CEFET-MG | DEPARTAMENTO DE COMPUTAÇÃO

# Computação Gráfica

## GLFW - Graphics Library Framework

Prof. Dr. Raulivan Rodrigo

# Contextualização

Biblioteca GLFW

Graphics Library Framework.

É uma biblioteca de código aberto, escrita na linguagem C, que serve como uma ponte entre o OpenGL e o Sistema Operacional

**pip install glfw**

**import glfw**

# GLFW - Graphics Library Framework

## Biblioteca GLFW

A maior limitação arquitetural do OpenGL: **O OpenGL é "cego e surdo" para o mundo externo**.

O OpenGL sabe fazer apenas **cálculos matemáticos na Placa de Vídeo para pintar pixels.**

- Ele não sabe o que é uma janela do Windows.
- Ele não sabe o que é um mouse, um teclado ou um joystick.
- Se você tiver apenas o OpenGL puro, você não consegue nem abrir uma tela preta no seu monitor.

# GLFW - Graphics Library Framework

## Biblioteca GLFW

O GLFW assume três responsabilidades fundamentais:

**Criação e Gerenciamento da Janela (01)**

- Cada Sistema Operacional desenha janelas de um jeito completamente diferente. O Windows usa a API Win32, o Linux usa X11 ou Wayland, o macOS usa Cocoa.
- O GLFW abstrai toda essa burocracia.
- Quando você chama **glfw.create_window()**, ele conversa com o SO nativo por debaixo dos panos e exige a criação de uma tela em branco onde a GPU possa trabalhar.

# GLFW - Graphics Library Framework

## Biblioteca GLFW

O GLFW assume três responsabilidades fundamentais:

**Criação do "Contexto" Gráfico (State Machine) (02)**

- Para a CPU enviar vértices para a GPU, eles precisam assinar um "contrato de comunicação" chamado de **Contexto OpenGL**.
- O GLFW negocia com o driver da sua placa de vídeo (NVIDIA, AMD ou Intel) pedindo a versão correta da tecnologia.

```
if not glfw.init(): return
glfw.window_hint(glfw.CONTEXT_VERSION_MAJOR, 3)
glfw.window_hint(glfw.CONTEXT_VERSION_MINOR, 3)
glfw.window_hint(glfw.OPENGL_PROFILE, glfw.OPENGL_CORE_PROFI
```

# GLFW - Graphics Library Framework

## Biblioteca GLFW

O GLFW assume três responsabilidades fundamentais:

**Leitura de Hardware (Input Handling) (03)**

- Como o OpenGL só desenha, é o GLFW que monitora a placa-mãe aguardando interrupções físicas de hardware.
- É ele que detecta que a tecla "Seta para Cima" foi pressionada ou que o mouse foi arrastado, enviando esses dados para o nosso laço de repetição no Python
- Através da função glfw.poll_events().

# GLFW - Graphics Library Framework

## Biblioteca GLFW

- A Placa de Vídeo (GPU) é o motor do carro.
- O OpenGL (e os Shaders) é a injeção eletrônica que faz o motor queimar combustível da forma mais eficiente possível.
- O GLFW é o chassi, o volante e os pedais.



# GLFW - Graphics Library Framework

## Na prática

- Instalar:

**pip install glfw PyOpenGL numpy**

- cubo_3d.py

cubo_3d.py U

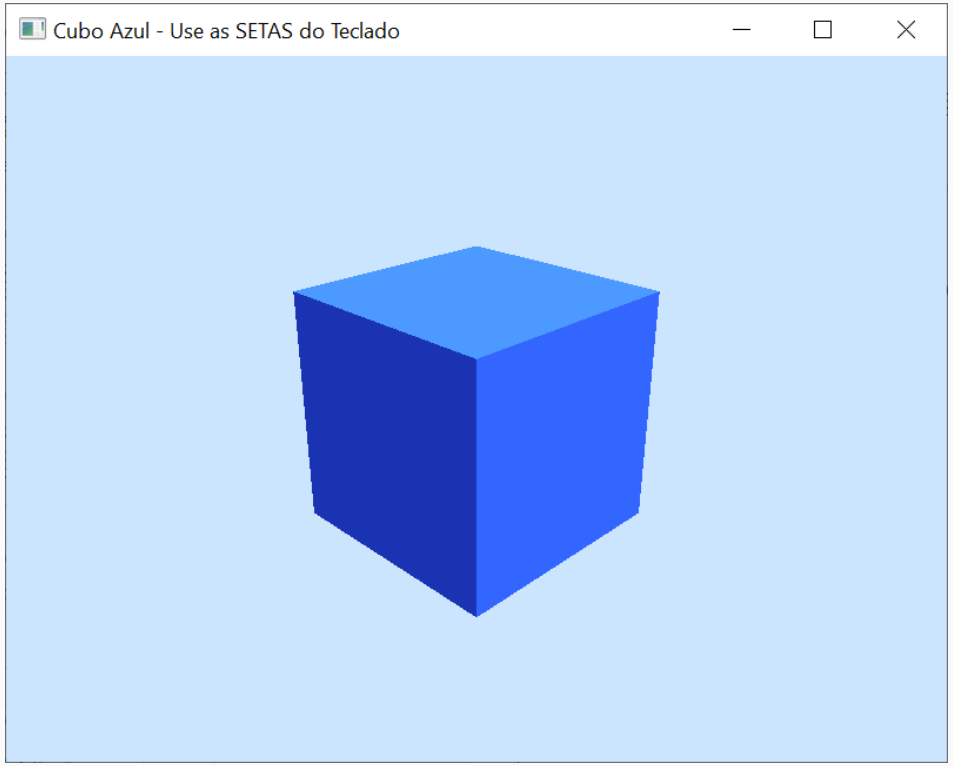
AULA14 > cubo_3d.py > ...

```
1   import glfw
2   from OpenGL.GL import *
3   import OpenGL.GL.shaders
4   import numpy as np
5
```

# GLFW - Graphics Library Framework

## Na prática

- Código: https://github.com/raulivan/computacao-grafica-python/blob/main/AULA15/cubo_3d.py



# GLFW - Graphics Library Framework

```
6   # SHADERS (A GPU pintando o azul)
7   VERTEX_SHADER = """
8   #version 330 core
9   layout (location = 0) in vec3 aPos;
10  layout (location = 1) in vec3 aColor;
11
12  out vec3 vertexColor;
13  uniform mat4 MVP;
14
15  void main() {
16      gl_Position = MVP * vec4(aPos, 1.0);
17      vertexColor = aColor; // Repassa o tom de azul para o fragmento
18  }
19  """
```

# GLFW - Graphics Library Framework

```
21    FRAGMENT_SHADER = """
22    #version 330 core
23    in vec3 vertexColor;
24    out vec4 FragColor;
25
26    void main() {
27        FragColor = vec4(vertexColor, 1.0); // Pinta a tela
28    }
29    """
```

## GLFW - Graphics Library Framework

```
31  def matriz_perspectiva(fov, aspecto, perto, longe):
32      f = 1.0 / np.tan(np.radians(fov) / 2.0)
33      m = np.zeros((4, 4), dtype=np.float32)
34      m[0, 0] = f / aspecto
35      m[1, 1] = f
36      m[2, 2] = (longe + perto) / (perto - longe)
37      m[2, 3] = (2.0 * longe * perto) / (perto - longe)
38      m[3, 2] = -1.0
39      return m
40      # fim matriz_perspectiva
```

GL

Na pr

- Inst
- cub

```
42  def matriz_translacao(x, y, z):
43      return np.array([
44          [1,0,0,x],
45          [0,1,0,y],
46          [0,0,1,z],
47          [0,0,0,1]]
48      , dtype=np.float32)
49      # fim matriz_translacao
50
51  def matriz_rotacao_x(graus):
52      c, s = np.cos(np.radians(graus)), np.sin(np.radians(graus))
53      return np.array([
54          [1,0,0,0],
55          [0,c,-s,0],
56          [0,s,c,0],
57          [0,0,0,1]],
58      dtype=np.float32)
59      # fim matriz_rotacao_x
60
```

# GLFW - Graphics Library Framework

**Na prática**

```
61  def matriz_rotacao_y(graus):
62      c, s = np.cos(np.radians(graus)), np.sin(np.radians(graus))
63      return np.array([
64          [c,0,s,0],
65          [0,1,0,0],
66          [-s,0,c,0],
67          [0,0,0,1]],
68      dtype=np.float32)
69      # fim matriz_rotacao_y
70
```

# GLFW - Graphics Library Framework

```
72  def main():
73
74      if not glfw.init(): return
75      glfw.window_hint(glfw.CONTEXT_VERSION_MAJOR, 3)
76      glfw.window_hint(glfw.CONTEXT_VERSION_MINOR, 3)
77      glfw.window_hint(glfw.OPENGL_PROFILE, glfw.OPENGL_CORE_PROFILE)
78
79      janela = glfw.create_window(800, 600, "Cubo Azul - Use as SETAS do Teclado", None, None)
80      if not janela:
81          glfw.terminate()
82          return
83
84      glfw.make_context_current(janela)
85      # VSync: Trava a execução em 60 FPS
86      glfw.swap_interval(1)
87      # O Z-Buffer que esconde as faces de trás
88      glEnable(GL_DEPTH_TEST)
89
```

# GLFW - Graphics Library Framework

## Na prática

- Instalar:

```
89
90      shader = OpenGL.GL.shaders.compileProgram(
91          OpenGL.GL.shaders.compileShader(VERTEX_SHADER, GL_VERTEX_SHADER),
92          OpenGL.GL.shaders.compileShader(FRAGMENT_SHADER, GL_FRAGMENT_SHADER)
93      )
94
```

# GLFW - Graphics Library Framework

```
95      # 24 Vértices (4 para cada uma das 6 faces)
96      # Formato: X, Y, Z,   R, G, B
97      vertices = np.array([
98          # Face Frontal (Azul Claro)
99          -0.5, -0.5,  0.5,   0.2, 0.4, 1.0,
100          0.5, -0.5,  0.5,   0.2, 0.4, 1.0,
101          0.5,  0.5,  0.5,   0.2, 0.4, 1.0,
102         -0.5,  0.5,  0.5,   0.2, 0.4, 1.0,
103
104         # Face Traseira (Azul Escuro)
105         -0.5, -0.5, -0.5,   0.0, 0.1, 0.5,
106          0.5, -0.5, -0.5,   0.0, 0.1, 0.5,
107          0.5,  0.5, -0.5,   0.0, 0.1, 0.5,
108         -0.5,  0.5, -0.5,   0.0, 0.1, 0.5,
109
110         # Face Esquerda (Azul Médio Escuro)
111         -0.5, -0.5, -0.5,   0.1, 0.2, 0.7,
112         -0.5, -0.5,  0.5,   0.1, 0.2, 0.7,
113         -0.5,  0.5,  0.5,   0.1, 0.2, 0.7,
114         -0.5,  0.5, -0.5,   0.1, 0.2, 0.7,
115
```

```
116         # Face Direita (Azul Médio Claro)
117          0.5, -0.5, -0.5,   0.15, 0.3, 0.9,
118          0.5, -0.5,  0.5,   0.15, 0.3, 0.9,
119          0.5,  0.5,  0.5,   0.15, 0.3, 0.9,
120          0.5,  0.5, -0.5,   0.15, 0.3, 0.9,
121
122         # Face Superior (Azul Brilhante)
123         -0.5,  0.5, -0.5,   0.3, 0.6, 1.0,
124          0.5,  0.5, -0.5,   0.3, 0.6, 1.0,
125          0.5,  0.5,  0.5,   0.3, 0.6, 1.0,
126         -0.5,  0.5,  0.5,   0.3, 0.6, 1.0,
127
128         # Face Inferior (Azul Quase Preto - Sombra)
129         -0.5, -0.5, -0.5,   0.0, 0.05, 0.2,
130          0.5, -0.5, -0.5,   0.0, 0.05, 0.2,
131          0.5, -0.5,  0.5,   0.0, 0.05, 0.2,
132         -0.5, -0.5,  0.5,   0.0, 0.05, 0.2,
133     ], dtype=np.float32)
```

# GLFW - Graphics Library Framework

```
135     # conectar os 24 vértices para formar 12 triângulos
136     indices = np.array([
137         0, 1, 2,  2, 3, 0,        # Frontal
138         4, 5, 6,  6, 7, 4,        # Traseira
139         8, 9, 10, 10, 11, 8,      # Esquerda
140         12, 13, 14, 14, 15, 12,   # Direita
141         16, 17, 18, 18, 19, 16,   # Superior
142         20, 21, 22, 22, 23, 20    # Inferior
143     ], dtype=np.uint32)
144
```

```
    # Memória da Placa de Vídeo)
    VAO = glGenVertexArrays(1)
    VBO = glGenBuffers(1)
    EBO = glGenBuffers(1)

    glBindVertexArray(VAO)

    glBindBuffer(GL_ARRAY_BUFFER, VBO)
    glBufferData(GL_ARRAY_BUFFER, vertices.nbytes, vertices, GL_STATIC_DRAW)

    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO)
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, indices.nbytes, indices, GL_STATIC_DRAW)

    passo = 6 * vertices.itemsize
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, passo, ctypes.c_void_p(0))
    glEnableVertexAttribArray(0)
    glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, passo, ctypes.c_void_p(3 * vertices.itemsize))
    glEnableVertexAttribArray(1)

    glBindVertexArray(0)
```

# GLFW - Graphics Library Framework

```
166     # matrizes Base
167     projecao = matriz_perspectiva(45.0, 800.0/600.0, 0.1, 100.0)
168     view = matriz_translacao(0, 0, -3.5) # Câmera recuada
169     loc_mvp = glGetUniformLocation(shader, "MVP")
170
171     # Variáveis de Estado da Rotação
172     rot_x = 25.0
173     rot_y = 45.0
174     velocidade = 2.0 # Velocidade do giro
175
176     print("Use as setas do teclado para rotacionar o cubo.")
177
```

# GLFW - Graphics Library Framework

```
179     # GAME LOOP (Laço Principal)
180     while not glfw.window_should_close(janela):
181         glfw.poll_events()
182 
183         # "A tecla está pressionada AGORA?"
184         if glfw.get_key(janela, glfw.KEY_RIGHT) == glfw.PRESS:
185             rot_y += velocidade
186         if glfw.get_key(janela, glfw.KEY_LEFT) == glfw.PRESS:
187             rot_y -= velocidade
188         if glfw.get_key(janela, glfw.KEY_DOWN) == glfw.PRESS:
189             rot_x += velocidade
190         if glfw.get_key(janela, glfw.KEY_UP) == glfw.PRESS:
191             rot_x -= velocidade
```

```
193     # Limpeza do frame anterior
194     glClearColor(0.8, 0.9, 1.0, 1.0) # Fundo azul bem claro
195     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
196     glUseProgram(shader)
197
198     # Atualização Matemática
199     model = matriz_rotacao_x(rot_x) @ matriz_rotacao_y(rot_y)
200     MVP = projecao @ view @ model
201
202     glUniformMatrix4fv(loc_mvp, 1, GL_FALSE,
203                        np.ascontiguousarray(MVP.T, dtype=np.float32))
204
205     # Desenho
206     glBindVertexArray(VAO)
207     glDrawElements(GL_TRIANGLES, len(indices), GL_UNSIGNED_INT, None)
208
209     glfw.swap_buffers(janela)
```

# GLFW - Graphics Library Framework

```
208
209             glfw.swap_buffers(janela)
210
211         glfw.terminate()
212
213     if __name__ == "__main__":
214         main()
```

# Dúvidas e Discussão

Este é o momento para aprofundarmos nos conceitos.

raulivan@cefetmg.br

SIGAA

# Vamos praticar

## Pirâmide e prisma

**Agora é sua vez, implemente a visualização 3D de uma pirâmide e de um prisma.**

# Próxima Aula

## Shaders

